AWS — Interview Ready Answers

Comprehensive answers with examples for all 33 AWS questions

33 Questions • EC2 • S3 • IAM • EKS • ECS • VPC • ALB • Lambda • RDS • Route53 • CloudWatch

---

Q1. What AWS services have you used in production?

Answer

This is an experience question — answer honestly based on your work, while demonstrating breadth and depth. Below is a comprehensive overview of commonly used AWS services in production DevOps environments.

Compute

- EC2 (Elastic Compute Cloud) — Virtual machines for running applications, web servers, and microservices.
- EKS (Elastic Kubernetes Service) — Managed Kubernetes for container orchestration in production.
- ECS (Elastic Container Service) — Container orchestration using Fargate (serverless) or EC2 launch types.
- Lambda — Serverless compute for event-driven workloads (API triggers, S3 events, scheduled jobs).

Storage & Database

- S3 (Simple Storage Service) — Object storage for artifacts, backups, static assets, and Terraform state.
- RDS — Managed relational databases (PostgreSQL, MySQL) with Multi-AZ for high availability.
- DynamoDB — NoSQL for low-latency, high-throughput workloads.
- EBS (Elastic Block Store) — Persistent block storage for EC2 and EKS StatefulSets.
- EFS (Elastic File System) — Shared NFS storage across multiple EC2/EKS pods.

Networking

- VPC — Custom isolated networks with public/private subnets, NAT Gateways, and Internet Gateways.
- ALB / NLB — Application and Network Load Balancers for routing traffic across services.
- Route 53 — DNS management, health checks, and geolocation-based routing.
- CloudFront — CDN for low-latency content delivery globally.

Security & Identity

- IAM — Roles, policies, and fine-grained access control for all AWS resources.
- Secrets Manager — Secure storage and automatic rotation of credentials.
- KMS — Key Management Service for encryption at rest and in transit.
- WAF — Web Application Firewall for protecting ALBs and CloudFront distributions.

Monitoring & Operations

- CloudWatch — Metrics, logs, alarms, and dashboards for infrastructure and application monitoring.
- CloudTrail — Audit logging for all API calls across the account.
- Systems Manager (SSM) — Patch management, parameter store, and remote access to EC2 without SSH.

CI/CD & Developer Tools

- CodePipeline / CodeBuild / CodeDeploy — Fully managed CI/CD pipeline for automated deployments.
- ECR (Elastic Container Registry) — Private Docker image registry integrated with EKS and ECS.

> *Tip: Always relate AWS services to a real use case — e.g., 'We used EKS with ALB Ingress Controller for our microservices, S3 for artifact storage, and CloudWatch for monitoring.'*

Q2. What is the purpose of a NAT Gateway?

Answer

A NAT (Network Address Translation) Gateway allows resources in a private subnet to initiate outbound connections to the internet — without exposing those resources to inbound connections from the internet.

Why It's Needed

In a secure AWS architecture, application servers, databases, and EKS worker nodes are placed in private subnets (no direct internet access). However, these resources still need to reach the internet for legitimate outbound traffic such as:

- Downloading OS patches and software packages (yum update, apt-get)
- Pulling Docker images from Docker Hub or other public registries

- Calling external APIs (payment gateways, third-party services)
- Sending logs or metrics to external monitoring tools

How It Works

The NAT Gateway is placed in a public subnet and has an Elastic IP. Private subnet resources route their internet-bound traffic to the NAT Gateway, which translates the private IP to its own public IP, sends the request, receives the response, and forwards it back to the originating private resource.

```
Private Subnet (EC2 / EKS Node)
  → Route Table: 0.0.0.0/0 → NAT Gateway
  → NAT Gateway (in Public Subnet, has Elastic IP)
  → Internet Gateway → Internet
```

NAT Gateway vs NAT Instance

Feature	NAT Gateway
Managed by	AWS fully managed
Availability	Highly available within AZ
Scalability	Auto-scales up to 45 Gbps
Cost	Higher (per hour + data)
Use case	Production

Best Practice

- Deploy one NAT Gateway per Availability Zone to avoid cross-AZ data transfer costs and eliminate single points of failure.
- Use VPC Endpoints (e.g., S3 Gateway Endpoint) to avoid routing S3 traffic through the NAT Gateway — significantly reduces data transfer costs.

> *Tip: If you notice high NAT Gateway costs, check if resources are pulling large files from S3 through NAT. Adding a free S3 Gateway VPC Endpoint routes that traffic directly and eliminates the cost.*

Q3. You have an application in Account A that needs to access an S3 bucket in Account B. How would you configure this?

Answer

Cross-account S3 access is a very common pattern in multi-account AWS environments. It requires granting permissions from BOTH sides — the IAM identity in Account A must have permission to call S3, and the S3 bucket in Account B must allow access from Account A.

Step 1 — Create an IAM Role or Policy in Account A

The application in Account A (running on EC2, EKS, Lambda, etc.) should have an IAM Role with a policy allowing it to access the S3 bucket in Account B:

```
{ "Effect": "Allow",
  "Action": ["s3:GetObject","s3:PutObject","s3:ListBucket"],
  "Resource": ["arn:aws:s3:::account-b-bucket","arn:aws:s3:::account-b-bucket/*"]
}
```

Step 2 — Add a Bucket Policy in Account B

The S3 bucket in Account B must have a bucket policy explicitly granting access to Account A's IAM Role or Account A's account ID:

```
{ "Principal": {"AWS": "arn:aws:iam::ACCOUNT_A_ID:role/app-role"},
  "Effect": "Allow",
  "Action": ["s3:GetObject","s3:PutObject","s3:ListBucket"],
  "Resource": ["arn:aws:s3:::account-b-bucket","arn:aws:s3:::account-b-bucket/*"]
}
```

Step 3 — Optional: Cross-Account Role Assumption

For more granular control, Account A's application can assume an IAM Role in Account B using STS AssumeRole, then use that role's temporary credentials to access the S3 bucket:

```
aws sts assume-role \
  --role-arn arn:aws:iam::ACCOUNT_B_ID:role/s3-access-role \
  --role-session-name cross-account-session
```

Important Considerations

- The bucket policy in Account B is mandatory — IAM policies in Account A alone are not sufficient for cross-account access.
- If using SSE-KMS encryption on the S3 bucket, the KMS key policy in Account B must also allow Account A to use the key (kms:Decrypt, kms:GenerateDataKey).
- For S3 object ownership: by default, objects uploaded by Account A are owned by Account A. Use 'BucketOwnerPreferred' or 'BucketOwnerEnforced' ACL to ensure Account B owns all objects.

> *Tip: Use aws:PrincipalOrgID condition in the bucket policy to restrict access to only accounts within your AWS Organization — a cleaner approach than listing individual account IDs.*

Q4. Your EC2 instance in a private subnet needs to download packages without NAT Gateway. What alternatives exist?

Answer

NAT Gateways cost money (per hour + per GB of data). Several alternatives can reduce or eliminate this cost depending on your use case.

Option 1 — VPC Endpoints (Best for AWS Services)

Interface VPC Endpoints or Gateway VPC Endpoints allow private communication with AWS services without going through the internet at all:

- S3 Gateway Endpoint — Free. Route S3 traffic directly from private subnet without NAT Gateway.
- ECR Interface Endpoint — Access ECR to pull Docker images privately.
- SSM Interface Endpoint — Use Systems Manager for patch management without internet.
- Secrets Manager, CloudWatch, EC2 API endpoints — All available as interface endpoints.

```
# Create S3 Gateway Endpoint (Free)
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-xxxx \
  --service-name com.amazonaws.us-east-1.s3 \
  --route-table-ids rtb-xxxx
```

Option 2 — NAT Instance (Cost-Effective for Low Traffic)

Launch a small EC2 instance in a public subnet configured to do NAT. Enable source/destination check disable on the instance. Cheaper for low-volume traffic but requires manual management and is a single point of failure.

Option 3 — AWS Systems Manager (SSM) for Patching

Instead of downloading packages from the internet, use SSM Patch Manager to apply OS patches. The SSM agent communicates with AWS via VPC endpoints — no internet access needed.

Option 4 — Private Package Mirror / Repository

Set up an internal package mirror (e.g., Nexus, Artifactory, or AWS CodeArtifact) inside the VPC. EC2 instances fetch packages from the internal mirror rather than the internet.

Option 5 — Pre-baked AMIs

Bake all required software into custom AMIs using Packer. EC2 instances launched from these AMIs don't need to download packages at runtime — everything is already installed.

Alternative	Best For
S3 / ECR VPC Endpoints	AWS service access
NAT Instance	Low internet traffic
SSM Patch Manager	OS patching only
Internal Repository	Custom packages, pip, npm
Custom AMIs (Packer)	Immutable infrastructure

> *Tip: Combine VPC endpoints for AWS services (S3, ECR, SSM) + CodeArtifact for package mirroring. This eliminates NAT Gateway entirely for most private subnet use cases.*

Q5. You need to import an existing AWS VPC into Terraform. What are the steps?

Answer

Importing existing AWS resources into Terraform allows you to manage infrastructure that was created manually (via Console or CLI) using Terraform going forward. It does NOT recreate the resource — it just links the existing resource to a Terraform state entry.

Step 1 — Write the Terraform Configuration

First, write a resource block in your .tf file that describes the existing VPC. The configuration must match what's actually deployed (CIDR, tags, etc.):

```
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = "production-vpc" }
}
```

Step 2 — Run terraform import

Use the terraform import command to link the existing VPC ID to the resource in your configuration:

```
terraform import aws_vpc.main vpc-0a1b2c3d4e5f67890
```

The format is: terraform import <resource_type>.<local_name> <remote_id>

Step 3 — Run terraform plan to Check for Drift

After importing, run terraform plan to see if there are any differences between your written configuration and the actual resource. If there are diffs, update your .tf file to match the real resource:

```
terraform plan
```

The goal is for terraform plan to show 'No changes' after the import is complete.

Step 4 — Import Dependent Resources

A VPC rarely exists alone. You must also import all associated resources: subnets, route tables, internet gateways, security groups, NAT gateways, etc. Each resource requires its own import command:

```
terraform import aws_subnet.public subnet-0a1b2c3d
terraform import aws_internet_gateway.igw igw-0a1b2c3d
terraform import aws_route_table.public rtb-0a1b2c3d
```

Modern Approach — import Block (Terraform 1.5+)

Terraform 1.5+ supports an import block in configuration files for a declarative, plan-friendly import:

```
import {
  to = aws_vpc.main
  id = "vpc-0a1b2c3d4e5f67890"
}
```

Common Pitfalls

- Computed attributes — Some attributes are set by AWS and not in your config. Terraform will show diffs for these. Use lifecycle { ignore_changes = [...] } for attributes you don't want to manage.
- State file — If using remote state (S3 + DynamoDB), ensure you're authenticated and state is locked before importing.
- Importing does not generate config — You must write the resource block manually (or use terraform plan -generate-config-out in v1.5+).

> *Tip: For large-scale imports, tools like Terraformer (github.com/GoogleCloudPlatform/terraformer) can auto-generate Terraform code and import state for entire AWS accounts.*

Q6. How would you implement cross-account resource provisioning using Terraform?

Answer

Cross-account provisioning with Terraform uses the AWS provider's assume_role feature to deploy resources into multiple AWS accounts from a single Terraform configuration or pipeline.

Approach 1 — Multiple Provider Configurations

Define multiple AWS providers with different IAM roles to deploy into different accounts simultaneously:

```
provider "aws" {
  alias  = "account_a"
  region = "us-east-1"
  assume_role { role_arn = "arn:aws:iam::ACCOUNT_A_ID:role/terraform-role" }
}
provider "aws" {
  alias  = "account_b"
```

```
  region = "us-east-1"
  assume_role { role_arn = "arn:aws:iam::ACCOUNT_B_ID:role/terraform-role" }
}
resource "aws_s3_bucket" "logs" {
  provider = aws.account_b
  bucket   = "my-logs-bucket"
}
```

Approach 2 — Separate Workspaces or Directories per Account

Maintain separate Terraform root modules for each account. Each has its own backend configuration pointing to different state files. A CI/CD pipeline loops through accounts and applies each in sequence.

IAM Role Requirements

Each target account must have an IAM Role that the Terraform pipeline can assume. The trust policy on that role must allow the pipeline's identity (e.g., GitHub Actions OIDC, central EC2 role) to assume it:

```
{ "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::CENTRAL_ACCOUNT:role/terraform-pipeline"},
  "Action": "sts:AssumeRole"
}
```

State Management

- Use a separate state file per account — e.g., S3 key: account-a/terraform.tfstate, account-b/terraform.tfstate.
- Centralise state in one S3 bucket in a management account, with DynamoDB locking.
- Restrict access to each state prefix using S3 bucket policies.

Real-World Pipeline Example

In GitHub Actions, use OIDC federation to assume an IAM role in each account, then run terraform apply per account:

```
for ACCOUNT in account-a account-b account-c; do
  cd environments/$ACCOUNT
  terraform init && terraform apply -auto-approve
done
```

> *Tip: Use AWS Organizations + Service Control Policies (SCPs) to restrict what the terraform-role can do in each account — ensuring the pipeline can only create approved resource types.*

Q7. An S3 bucket was created via Terraform, but someone manually added a policy. How do you handle this drift?

Answer

This is a Terraform state drift scenario. The actual infrastructure (S3 bucket policy) differs from what Terraform knows about in its state file. Here's how to handle it properly.

Step 1 — Detect the Drift

```
terraform plan
```

Terraform will show the manually added policy as a change it wants to remove or overwrite, because the state file doesn't know about it. This confirms drift.

```
terraform plan -refresh-only
```

This shows what changed in the real infrastructure without planning any changes — helpful for understanding the scope of drift.

Step 2 — Evaluate the Manual Change

Before doing anything, understand the intent of the manual change. Ask: Was this policy added intentionally as a permanent change? Or was it an accident/test?

Option A — Incorporate the change into Terraform code (Recommended)

If the manually added policy is valid and should be permanent, add it to your Terraform configuration to match the real state:

```
resource "aws_s3_bucket_policy" "bucket_policy" {
  bucket = aws_s3_bucket.my_bucket.id
  policy = jsonencode({
    Statement = [{
      Effect    = "Allow"
      Principal = { AWS = "arn:aws:iam::123456789:root" }
      Action    = "s3:GetObject"
      Resource  = "${aws_s3_bucket.my_bucket.arn}/*"
    }]
```

```
  })
}
```

After updating the code, run terraform plan — it should now show no changes.

Option B — Revert the manual change (Let Terraform win)

If the change was accidental or invalid, run terraform apply. Terraform will overwrite the manually added policy with the state it manages. Communicate to the team WHY this happened and enforce the 'no manual changes to Terraform-managed resources' policy.

Option C — Use lifecycle ignore_changes (Use sparingly)

If the policy is managed by a separate process (e.g., another team manages the bucket policy separately), instruct Terraform to ignore changes to that attribute:

```
resource "aws_s3_bucket_policy" "bucket_policy" {
  lifecycle { ignore_changes = [policy] }
}
```

Prevention — Stop Drift Before It Happens

- Enforce 'IaC only' changes using SCPs or IAM permission boundaries — prevent humans from modifying Terraform-managed resources via Console.
- Run terraform plan -refresh-only in your CI/CD pipeline nightly to detect drift early.
- Use AWS Config rules to alert when S3 bucket policies are changed outside of Terraform.
- Tag all Terraform-managed resources with ManagedBy=Terraform and set up alarms on untagged changes.

> *Tip: Drift is inevitable in teams where multiple people have console access. Build drift detection into your pipeline — not just your incident response.*

Q8. Your company's cloud costs are increasing rapidly. How would you approach cost optimization without impacting performance?

Answer

Cloud cost optimization is a continuous discipline, not a one-time exercise. Approach it systematically across compute, storage, networking, and architectural patterns.

Step 1 — Gain Visibility First

- Enable AWS Cost Explorer and set up Cost Allocation Tags (Environment, Team, Service) on every resource.
- Use AWS Cost and Usage Reports (CUR) for granular billing data.
- Set up AWS Budgets with alerts at 80% and 100% of monthly targets.
- Use AWS Compute Optimizer to get right-sizing recommendations for EC2, EKS, Lambda.

Step 2 — Right-Size Compute

- EC2: Identify over-provisioned instances using CloudWatch CPU/memory metrics. Downsize where utilization < 20%.
- EKS: Set correct resource requests and limits on pods. Use Vertical Pod Autoscaler (VPA) recommendations.
- RDS: Switch to serverless or smaller instance types for non-production databases.

Step 3 — Use Savings Plans and Reserved Instances

- Commit to 1-year or 3-year Savings Plans for predictable baseline workloads — savings of 30-72% vs On-Demand.
- Use Spot Instances for stateless, fault-tolerant workloads (batch processing, CI/CD runners, dev/test environments).

```
# EKS mixed node group: On-Demand for critical pods, Spot for stateless
node_groups:
  on_demand: { instance_types: [m5.large], min_size: 2 }
  spot:      { instance_types: [m5.large, m5.xlarge], min_size: 0 }
```

Step 4 — Optimize Storage

- S3: Apply lifecycle policies to transition objects to S3-IA (Infrequent Access) or Glacier after 30/90 days.
- EBS: Delete unattached EBS volumes. Switch gp2 volumes to gp3 (same performance, 20% cheaper).
- RDS: Enable storage auto-scaling to avoid over-provisioning.

Step 5 — Reduce Data Transfer Costs

- Add S3 and DynamoDB VPC Gateway Endpoints — routes traffic within AWS network, eliminates NAT Gateway charges for that traffic.
- Use CloudFront to cache responses — reduces origin requests and data transfer costs.
- Place services in the same AZ when they communicate heavily — avoids cross-AZ data transfer fees.

Step 6 — Auto-Scaling and Scheduling

- Use HPA and Cluster Autoscaler in EKS — scale to zero during off-hours for non-production environments.

- Schedule Lambda or EventBridge rules to stop RDS and EC2 instances in dev/test overnight.
- Use AWS Instance Scheduler to automate start/stop of non-production resources.

Step 7 — Architectural Review

- Replace always-on EC2 with Lambda for infrequent workloads.
- Use SQS + Lambda instead of dedicated EC2 workers for queue processing.
- Review NAT Gateway usage — high NAT costs often indicate mis-routed S3 or ECR traffic that should use VPC Endpoints.

> *Tip: Start with visibility (tagging + Cost Explorer), then right-sizing, then Savings Plans. Don't commit to Reserved Instances before right-sizing — you'll commit to the wrong instance type.*

Q9. How would you set up geolocation-based routing using AWS services?

Answer

AWS offers multiple services for geolocation-based routing depending on the use case — Route 53 for DNS-level routing and CloudFront for content delivery with geo-targeting.

Option 1 — Route 53 Geolocation Routing

Route 53 Geolocation routing routes DNS queries based on the geographic location of the user (continent, country, or US state). You create multiple resource record sets for the same domain name, each with a different geographic origin.

```
# Example: Route EU users to EU servers, US users to US servers
Record Set 1: myapp.com → EU-ALB (Location: Europe)
Record Set 2: myapp.com → US-ALB (Location: North America)
Record Set 3: myapp.com → Default-ALB (Location: Default – catch-all)
```

Configuration steps:

- Create ALBs or EC2 instances in each target region.
- In Route 53, create a Hosted Zone for your domain.
- Add A/CNAME records with Routing Policy = Geolocation. Select the continent/country for each.
- Add a default record for all other locations.
- Attach Route 53 Health Checks to automatically failover if a regional endpoint goes down.

Option 2 — Route 53 Geoproximity Routing

Routes traffic based on the geographic proximity of users to your resources, with the ability to bias routing to specific regions. Useful when you want to shift more traffic to a new region gradually.

Option 3 — CloudFront with Geo-Restriction

CloudFront can serve different content or block access based on country. Use Lambda@Edge or CloudFront Functions to inspect the CloudFront-Viewer-Country header and route or block accordingly:

```
// CloudFront Function – redirect EU users to EU origin
if (request.headers['cloudfront-viewer-country'].value === 'DE') {
  request.origin.custom.domainName = 'eu-api.myapp.com';
}
```

Option 4 — Global Accelerator

AWS Global Accelerator routes traffic to the nearest healthy regional endpoint using the AWS global backbone network rather than the public internet. Provides better latency and automatic failover across regions.

Service	Best For
Route 53 Geolocation	Country/continent-based DNS routing
Route 53 Geoproximity	Distance-based with bias control
CloudFront + Functions	Content customisation per country
Global Accelerator	Lowest latency, multi-region HA

> *Tip: Always add a 'Default' record in Route 53 geolocation routing — without it, users from unmatched locations will get NXDOMAIN (DNS resolution failure) instead of being routed to any endpoint.*

Q10. How do you structure Terraform code for a multi-region, multi-account AWS setup?

Answer

A well-structured multi-region, multi-account Terraform setup promotes code reuse, isolates blast radius, and scales cleanly as your organization grows.

Recommended Directory Structure

```
infrastructure/
├── modules/              # Reusable modules (vpc, eks, rds, etc.)
│   ├── vpc/
│   ├── eks/
│   └── rds/
├── accounts/
│   ├── prod/
│   │   ├── us-east-1/   # Per-region root module
│   │   │   ├── main.tf
│   │   │   ├── variables.tf
│   │   │   ├── terraform.tfvars
│   │   │   └── backend.tf
│   │   └── eu-west-1/
│   ├── staging/
│   └── dev/
```

Key Design Principles

- Separate state per account + region — Each account/region has its own S3 backend key (e.g., prod/us-east-1/terraform.tfstate). This prevents a failed apply in one region from affecting another.
- Reusable modules — The VPC, EKS cluster, and RDS modules are written once. Each account/region calls the module with different variable values (instance sizes, CIDR ranges, etc.).
- Provider configuration — Each root module uses assume_role to deploy into the correct account:

```
provider "aws" {
  region = var.region
  assume_role { role_arn = var.terraform_role_arn }
}
```

State Backend Configuration

```
terraform {
  backend "s3" {
    bucket         = "central-terraform-state"
    key            = "prod/us-east-1/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-state-lock"
    encrypt        = true
  }
}
```

CI/CD Pipeline

Each account+region combination is an independent pipeline run. A matrix strategy in GitHub Actions or a loop in a shell script can iterate over all combinations:

```
matrix:
  include:
    - account: prod    region: us-east-1
    - account: prod    region: eu-west-1
    - account: staging region: us-east-1
```

> *Tip: Use Terragrunt for keeping backend configuration DRY across many account/region combinations. It auto-generates backend config from a single root terragrunt.hcl file.*

Q11. What is your approach to tagging and cost governance across cloud infrastructure?

Answer

Tagging is the foundation of cloud cost governance. Without consistent tags, cost attribution becomes impossible and optimization efforts are guesswork.

Mandatory Tag Taxonomy

Define a minimum set of tags that every resource must have:

Tag Key	Example Values
Environment	prod, staging, dev

Team	platform, backend, data
Service	api-gateway, payments
ManagedBy	terraform, manual
CostCenter	CC-1234
Project	project-alpha

Enforcement — AWS Config Rules

Use AWS Config managed rules to detect and alert on resources missing mandatory tags:

- required-tags — Checks EC2, S3, RDS, Lambda, etc. for specified tag keys.
- Auto-remediation via SSM Automation — Automatically tag or alert when non-compliant resources are found.

Terraform Enforcement

Add a default_tags block in the AWS provider to apply tags to every resource automatically:

```
provider "aws" {
  default_tags {
    tags = {
      ManagedBy   = "terraform"
      Environment = var.environment
      Team        = var.team
    }
  }
}
```

Cost Governance Actions

- Enable Cost Allocation Tags in the Billing console — only tagged resources appear in Cost Explorer breakdowns.
- Set up AWS Budgets per team/environment with SNS alerts at 80% and 100% thresholds.
- Use Cost Explorer to identify top cost drivers by tag each week.
- Create showback/chargeback reports per team using Cost and Usage Reports (CUR) + Athena.
- Use AWS Organizations Tag Policies to enforce tag key naming conventions across all accounts.

Governance Workflow

- Infra PR creates new resources → Terraform plan includes tags → Reviewed in PR.
- AWS Config rule fires if resource created without mandatory tags → SNS alert to team → Ticket created.
- Monthly cost review: Cost Explorer report by Team tag shared with each team lead.

> *Tip: Make tags a PR requirement — add a CI check (e.g., using tflint or custom OPA policy) that fails the pipeline if any resource is missing mandatory tags.*

Q12. How do you rotate AWS IAM credentials and ensure services update them dynamically?

Answer

Credential rotation is a critical security practice. The approach differs based on whether you're dealing with IAM User access keys (static credentials) or IAM Roles (temporary credentials — the preferred approach).

Best Practice — Use IAM Roles Instead of Access Keys

IAM Roles issue short-lived temporary credentials (via STS) that auto-expire and auto-rotate. Services running on EC2, EKS (via IRSA), Lambda, and ECS should ALWAYS use roles — never long-lived access keys.

- EC2: Attach an instance profile. The AWS SDK reads credentials from the metadata endpoint automatically.
- EKS: Use IRSA (IAM Roles for Service Accounts) — pods get role credentials via projected service account tokens.
- Lambda: Assign an execution role. Lambda auto-rotates the STS credentials.
- ECS: Use Task Roles — credentials are injected into the container via the ECS credential provider.

Rotating IAM User Access Keys (When Roles Aren't Possible)

For legacy systems or third-party tools that require static access keys, follow this rotation process:

- Step 1: Create a new access key for the IAM user (each user can have max 2 active keys at once).
- Step 2: Update the application/service configuration with the new key.
- Step 3: Verify the application works with the new key.
- Step 4: Deactivate (not delete) the old key. Monitor for any errors.
- Step 5: Delete the old key after a safe window (e.g., 24-48 hours).

```
# Step 1: Create new key
```

```
aws iam create-access-key --user-name my-service-user
# Step 4: Deactivate old key
aws iam update-access-key --access-key-id OLD_KEY_ID --status Inactive
# Step 5: Delete old key
aws iam delete-access-key --access-key-id OLD_KEY_ID
```

Automating Rotation with AWS Secrets Manager

Store access keys in AWS Secrets Manager and configure automatic rotation. A Lambda function performs the create-update-verify-delete cycle automatically on a schedule:

```
aws secretsmanager rotate-secret \
  --secret-id iam-access-key-secret \
  --rotation-rules AutomaticallyAfterDays=30
```

Ensuring Services Pick Up New Credentials

- For applications using Secrets Manager: implement credential retrieval at connection time, not startup time. Use the caching SDK to refresh credentials before expiry.
- For Kubernetes: use External Secrets Operator to sync Secrets Manager values to K8s Secrets and trigger pod restarts on rotation.
- For environment variables: avoid — they're set at startup and don't update. Use SDK-based retrieval instead.

> *Tip: Run 'aws iam generate-credential-report' and check the 'access_key_last_rotated' column to identify keys not rotated in the last 90 days. Automate this check in your security pipeline.*

Q13. Design a highly available, scalable architecture for a microservices application on AWS with auto-scaling and disaster recovery.

Answer

A production-grade microservices architecture on AWS is designed around multi-AZ redundancy, auto-scaling at every layer, and clear DR objectives (RTO/RPO).

Core Architecture — Multi-AZ, Multi-Service

```
Internet → Route 53 → CloudFront → WAF
  → ALB (Multi-AZ) → EKS Cluster
  → Services: Auth | Orders | Payments | Notifications
  → RDS Multi-AZ | ElastiCache | SQS
  → S3 | ECR | Secrets Manager
```

Compute Layer — EKS

- EKS cluster spanning 3 Availability Zones with managed node groups.
- Horizontal Pod Autoscaler (HPA) scales pods based on CPU/memory or custom metrics (requests per second).
- Cluster Autoscaler or Karpenter auto-scales EC2 nodes based on pending pod demand.
- Mix of On-Demand (critical services) and Spot instances (stateless workers) for cost efficiency.

Networking

- VPC with public subnets (ALB, NAT Gateway) and private subnets (EKS nodes, RDS, ElastiCache).
- ALB with path-based routing across services. Health checks on /health endpoints.
- VPC endpoints for S3, ECR, Secrets Manager — no NAT Gateway costs for AWS service traffic.

Data Layer

- RDS PostgreSQL with Multi-AZ deployment (automatic failover in <60 seconds).
- Read replicas for scaling read traffic. RDS Proxy for connection pooling.
- ElastiCache Redis (cluster mode) for session storage and caching.
- SQS for async communication between services — decouples and absorbs traffic spikes.

Auto-Scaling Strategy

Layer	Scaling Mechanism
Pods (EKS)	HPA
Nodes (EC2)	Cluster Autoscaler / Karpenter
ALB	Auto-scales transparently
RDS Read Replicas	Manual or custom automation

Disaster Recovery

- RTO (Recovery Time Objective): < 15 minutes. RPO (Recovery Point Objective): < 5 minutes.
- RDS automated backups + Point-in-Time Recovery (PITR). Cross-region snapshot copies for DR.
- S3 Cross-Region Replication for critical data. Versioning enabled.
- Terraform IaC — entire infrastructure can be re-provisioned in a new region within minutes.
- Route 53 health checks + failover routing — automatic DNS failover to DR region.
- DR environment kept warm (pilot light) in a secondary region with minimal resources running.

Observability

- CloudWatch Container Insights for EKS metrics. Prometheus + Grafana for custom dashboards.
- Centralized logging: Fluent Bit → CloudWatch Logs or OpenSearch.
- X-Ray distributed tracing for inter-service call visibility.
- PagerDuty integration for on-call alerting on SLO breaches.

> *Tip: Define SLOs per service (e.g., 99.9% availability, p99 latency < 300ms). Alert on error budget burn rate, not just raw error count — this reduces alert fatigue and keeps focus on user impact.*

Q14. Design a VPC with multiple subnets, security groups, and explain the networking concepts for a production workload.

Answer

A production VPC is designed with defense-in-depth: public-facing resources in public subnets, application servers in private subnets, and databases in isolated subnets — each layer protected by security groups.

VPC CIDR and Subnet Design

```
VPC CIDR: 10.0.0.0/16 (65,536 IPs)

Public Subnets (ALB, NAT Gateway, Bastion):
  10.0.1.0/24 — AZ-a
  10.0.2.0/24 — AZ-b
  10.0.3.0/24 — AZ-c

Private App Subnets (EKS Nodes, EC2 App Servers):
  10.0.11.0/24 — AZ-a
  10.0.12.0/24 — AZ-b
  10.0.13.0/24 — AZ-c

Private DB Subnets (RDS, ElastiCache — no internet access):
  10.0.21.0/24 — AZ-a
  10.0.22.0/24 — AZ-b
  10.0.23.0/24 — AZ-c
```

Routing

- Public subnets: route table has 0.0.0.0/0 → Internet Gateway. Resources have public IPs.
- Private app subnets: route table has 0.0.0.0/0 → NAT Gateway (in public subnet). Resources have private IPs only.
- DB subnets: no route to internet. Only accessible from app subnets.

Security Group Design (Layered)

Security Group	Inbound Rules
alb-sg	443 from 0.0.0.0/0 (HTTPS)
app-sg	8080 from alb-sg only
db-sg	5432 from app-sg only
bastion-sg	22 from office IP only

Key Networking Concepts

- Internet Gateway (IGW): Attached to VPC. Enables bidirectional internet access for public subnet resources.
- NAT Gateway: In public subnet. Enables outbound-only internet access for private resources. One per AZ for HA.
- VPC Peering: Connect two VPCs for private communication. Non-transitive — A peers with B and C, but B cannot reach C via A.
- VPC Endpoints: Private connections to AWS services (S3, ECR, SSM) without internet traffic.

- NACLs (Network ACLs): Stateless subnet-level firewall. Applied before security groups. Rules evaluated in number order.
- Security Groups: Stateful instance-level firewall. Return traffic is automatically allowed.

Key Difference — Security Groups vs NACLs

Aspect	Security Groups
State	Stateful (return traffic auto-allowed)
Level	Instance/ENI level
Rule evaluation	All rules evaluated
Default	Deny all inbound, allow all outbound

> *Tip: Always use security groups as the primary control mechanism. Use NACLs only for broad subnet-level blocking (e.g., block an entire IP range known for attacks). Managing both becomes complex and error-prone.*

Q15. How would you implement automated upgrades for an EKS cluster with zero downtime?

Answer

EKS cluster upgrades involve upgrading the control plane, managed node groups, add-ons, and Kubernetes manifests. Done correctly, the entire process is zero-downtime.

Step 1 — Upgrade the Control Plane

The control plane is upgraded first. AWS manages this — it's a one-click operation in the console or a single API call:

```
aws eks update-cluster-version \
  --name my-cluster \
  --kubernetes-version 1.30
```

AWS upgrades control plane components (API server, etcd, scheduler) in a rolling manner with no downtime to running workloads. This typically takes 20-45 minutes.

Step 2 — Update EKS Add-Ons

After the control plane, update add-ons (CoreDNS, kube-proxy, vpc-cni, EBS CSI driver) to versions compatible with the new Kubernetes version:

```
aws eks update-addon --cluster-name my-cluster \
  --addon-name coredns --addon-version v1.11.1-eksbuild.9
```

Step 3 — Upgrade Managed Node Groups (Rolling)

For managed node groups, AWS performs a rolling node upgrade — draining one node at a time, launching a new node with the updated AMI, and only proceeding when the new node is healthy:

```
aws eks update-nodegroup-version \
  --cluster-name my-cluster \
  --nodegroup-name prod-nodes
```

Step 4 — Ensure Zero-Downtime During Node Drain

To guarantee zero downtime during node draining, your application pods must have:

- Pod Disruption Budgets (PDB) — prevents draining a node if it would violate minimum availability.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
spec:
  minAvailable: 2       # always keep at least 2 pods running
  selector:
    matchLabels: { app: my-service }
```

- Multiple replicas — always run at least 2 replicas for each critical service, spread across AZs using topologySpreadConstraints.
- Readiness probes — ensure traffic is only sent to pods that are ready.
- terminationGracePeriodSeconds — give pods time to finish in-flight requests before shutdown.

Step 5 — Update Application Manifests if Needed

Check for deprecated Kubernetes API versions in your manifests. Use kubectl convert or tools like pluto to detect deprecated APIs before upgrading.

Pre-Upgrade Checklist

- Test the upgrade in a staging cluster first.
- Back up etcd (done automatically by AWS for EKS, but keep your manifests in Git).

- Review the Kubernetes changelog for breaking changes between versions.
- Check all Helm charts and operators for compatibility with the new version.

> *Tip: Upgrade Kubernetes one minor version at a time (e.g., 1.27 → 1.28 → 1.29). Skipping versions is not supported and can cause compatibility issues.*

Q16. How do you implement security best practices in AWS networking and IAM policies?

Answer

AWS security is built on two pillars: network isolation (who can reach what) and identity/access control (who can do what). Both must be implemented with least-privilege as the guiding principle.

Network Security Best Practices

- Private subnets by default — Place all application servers, databases, and EKS nodes in private subnets. Only load balancers belong in public subnets.
- Security group rules by SG reference, not IP — Allow traffic from specific security groups rather than IP ranges. This is more maintainable and survives IP changes.
- VPC Flow Logs — Enable on all VPCs to capture network traffic metadata for security analysis and incident response.
- AWS Network Firewall or WAF — Deploy at the perimeter to filter malicious traffic before it reaches your ALB.
- No 0.0.0.0/0 inbound on SSH (port 22) — Use SSM Session Manager for EC2 access instead of opening SSH to the internet.
- VPC Endpoints — Keep AWS service traffic (S3, ECR, SSM) within the AWS network. Prevents traffic from traversing the internet.

IAM Security Best Practices

- Principle of Least Privilege — Grant only the minimum permissions required. Start with deny-all and add specific allows.

```
{ "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": "arn:aws:s3:::my-bucket/app-data/*"
}
```

- No root account usage — Lock the root account with MFA. Create IAM users or use SSO for all human access.
- IAM Roles over IAM Users — Use roles for all services (EC2, Lambda, EKS). Never use long-lived access keys for services.
- Resource-based conditions — Use conditions to restrict actions by IP, MFA, service, time window:

```
{ "Condition": { "Bool": { "aws:MultiFactorAuthPresent": "true" } } }
```

- Service Control Policies (SCPs) — Apply organization-level guardrails. Example: prevent disabling CloudTrail, restrict to approved regions.
- IAM Access Analyzer — Continuously analyze policies to detect overly permissive access, including cross-account and public resource exposure.
- Permission Boundaries — Set maximum permissions for IAM roles created by developers, preventing privilege escalation.

Monitoring and Detection

- CloudTrail — Log all API calls. Send to CloudWatch Logs and set alarms on sensitive actions (root login, IAM changes, Security Group changes).
- GuardDuty — Enable in all regions. Detects threat patterns like credential compromise, port scanning, cryptomining.
- Security Hub — Aggregates findings from GuardDuty, Inspector, Macie, Config into a unified security posture dashboard.
- AWS Config — Continuously evaluates resource configurations against compliance rules. Auto-remediate violations.

> *Tip: Run the AWS Trusted Advisor Security checks weekly. It flags publicly accessible S3 buckets, security groups with unrestricted access, and IAM users without MFA automatically.*

Q17. How would you implement a centralized logging solution across multiple cloud platforms and environments?

Answer

Centralized logging provides a single pane of glass for all application and infrastructure logs, enabling faster debugging, security analysis, and compliance reporting across environments and accounts.

Architecture — AWS-Centric Multi-Account

```
All Accounts (prod, staging, dev)
  → Fluent Bit / Fluentd (on every node/pod)
  → Central Logging Account
     → CloudWatch Logs (short-term, hot)
     → S3 (long-term, cold archival)
     → OpenSearch (search and dashboards)
```

Log Collection Layer

- EKS: Deploy Fluent Bit as a DaemonSet. It reads container logs from /var/log/containers/ and ships to CloudWatch Logs or directly to OpenSearch.
- EC2: CloudWatch Agent installed via SSM. Collects OS logs, application logs, and custom metrics.
- Lambda: Automatically sends logs to CloudWatch Logs — no agent needed.
- CloudTrail: API audit logs centralized to a dedicated S3 bucket in the logging account.

Cross-Account Log Aggregation

Create a dedicated Logging Account in your AWS Organization. Configure log destinations with resource policies that allow all org accounts to write:

```
# CloudWatch Log destination in logging account
aws logs put-destination-policy \
  --destination-name central-logs \
  --access-policy '{"Principal":{"AWS":"*"},"Condition":{"StringEquals":
{"aws:PrincipalOrgID":"o-xxxx"}}}'
```

Multi-Cloud (Azure / GCP)

- Azure: Use Diagnostic Settings to route logs to Event Hub → Lambda → CloudWatch / OpenSearch.
- GCP: Use Pub/Sub → Dataflow or a Forwarder → CloudWatch Logs.
- Alternatively: ship all clouds to a cloud-agnostic platform like Datadog, Elastic Cloud, or Grafana Loki.

Log Retention and Archival

- CloudWatch Logs: 30-day retention for hot logs (searchable quickly). Auto-export to S3 after 30 days.
- S3: Lifecycle rules to transition logs to S3-IA after 30 days, Glacier after 90 days. Retain 1 year for compliance.
- Enable S3 Object Lock (WORM) for audit/compliance logs — prevents deletion even by admins.

Alerting on Logs

Use CloudWatch Metric Filters to create metrics from log patterns and trigger alarms:

```
# Alert on 5xx errors from ALB access logs
filter pattern: '[status=5*]'
→ CloudWatch Alarm → SNS → PagerDuty
```

> *Tip: Standardize log format across all services using structured JSON with common fields: timestamp, level, service, traceId, userId. This makes cross-service correlation and querying dramatically easier.*

Q18. How do you ensure cost-efficient auto-scaling of infrastructure in cloud when managing high workloads in CI/CD?

Answer

Auto-scaling for CI/CD workloads requires a different strategy than production application scaling — builds are bursty, ephemeral, and often predictable in timing.

CI/CD Runner Auto-Scaling on AWS

- GitHub Actions self-hosted runners: Use GitHub's Actions Runner Controller (ARC) on EKS. Runners scale from 0 to N automatically based on queued jobs, then terminate when idle.
- Jenkins: Use the EC2 Fleet Plugin or Kubernetes Plugin to spin up build agents on-demand and terminate after build completion.
- GitLab CI: Use the GitLab Runner with Docker Machine or Kubernetes executor for dynamic scaling.

Cost Optimization for CI/CD Compute

- Spot Instances — CI/CD builds are perfect for Spot (interruptible) — if a build is interrupted, simply retry. Savings: 60-90% vs On-Demand.

```
# EKS node group for CI/CD runners — spot only
nodeGroups:
  - name: ci-runners
    instanceTypes: [m5.xlarge, m5.2xlarge, m4.xlarge]
    spot: true
```

```
    minSize: 0
    maxSize: 50
```

- Scale-to-zero — CI/CD nodes should terminate when there are no queued jobs. Configure minSize=0 for CI node groups.
- Right-size build containers — Profile actual build resource usage and set appropriate CPU/memory requests. Over-provisioned build containers waste money.

Caching to Reduce Build Time and Cost

- Cache Docker layers in ECR — Use BuildKit inline cache or an external cache registry.
- Cache dependencies (npm, pip, Maven) — Use S3 as a cache backend for GitHub Actions or store in EFS mounted by runners.
- Shorter builds = fewer compute minutes = lower cost. A 30-minute build on 4 runners costs 8x a 15-minute build on 2 runners.

EKS Cluster Autoscaling for CI/CD

- Karpenter (preferred): Launches nodes within 30-60 seconds of a pending pod. Bin-packs efficiently and uses Spot by default. Terminates nodes when workload ends.
- Separate node groups: Keep CI/CD runners on separate node groups from production workloads — different instance types, Spot vs On-Demand, different taints.

Monitoring CI/CD Costs

- Tag all CI/CD resources with Service=cicd, Team=platform.
- Set up a dedicated AWS Budget for CI/CD infrastructure.
- Track build minutes per repository — identify expensive builds for optimization.

> *Tip: The biggest CI/CD cost killer is idle runners. Implement aggressive scale-to-zero and short idle timeouts (2-5 minutes). A runner sitting idle for hours costs the same as one actively building.*

Q19. Explain a scenario where you had to design a disaster recovery (DR) strategy for DevOps infrastructure.

Answer

A DR strategy for DevOps infrastructure is not just about the applications — it includes the CI/CD pipelines, container registries, secrets, monitoring, and state files themselves. Here's a real-world scenario and approach.

Scenario

A fintech company's primary AWS region (us-east-1) becomes partially unavailable due to an AZ-wide outage affecting their EKS cluster, RDS database, and Jenkins build servers. The target is: RTO of 30 minutes, RPO of 15 minutes.

DR Architecture Designed

- Multi-region active-passive setup: us-east-1 (primary), eu-west-1 (DR). DR region kept warm (pilot light) — VPC, EKS cluster skeleton, and networking pre-provisioned.
- Route 53 health checks on primary ALB. Failover record points to DR region ALB. DNS TTL set to 60 seconds.

Data Replication Strategy

- RDS: Enabled cross-region read replica in eu-west-1. RPO = seconds (continuous replication). During failover, promote replica to standalone primary.
- S3 artifacts and Terraform state: Cross-Region Replication enabled. All buckets replicated in near real-time.
- ECR images: Use ECR replication rules to automatically push images to eu-west-1 ECR.
- Secrets Manager: Replicated to DR region using Secrets Manager cross-region replication.

CI/CD DR

- GitHub Actions: No regional dependency — pipelines continue to run. Point deploy step to DR region endpoint.
- Terraform state: Stored in S3 with CRR. DR pipelines point to DR state bucket. State locking DynamoDB table replicated via DynamoDB Global Tables.

Failover Runbook

```
1. PagerDuty alert: primary region degraded
2. On-call engineer confirms outage via CloudWatch dashboard
3. Promote RDS read replica in eu-west-1 to primary (5-10 min)
4. Scale up EKS DR node group (pre-warmed, starts in 3-5 min)
5. Update Kubernetes manifests to use DR region endpoints
6. Route 53 health check triggers automatic DNS failover
7. Verify application health in DR region
8. Total time: ~15-20 minutes
```

Testing

- Conduct quarterly DR drills — simulate primary region failure and execute full failover.
- Chaos Engineering: Use AWS Fault Injection Simulator to test individual component failures.

> *Tip: DR only works if it's tested regularly. An untested DR plan is essentially no DR plan. Schedule quarterly failover drills and measure actual RTO vs target.*

Q20. How do you manage and connect services like DBs, EC2, EKS, or ECS? Include the command to connect to ECS.

Answer

Managing connectivity between AWS services involves the right combination of networking (VPC, security groups), authentication (IAM roles), and service discovery (DNS, service endpoints).

Connecting Application to RDS

- Place RDS in a private subnet. Security group allows inbound 5432/3306 only from the application's security group.
- Store DB credentials in Secrets Manager. Application retrieves them at startup using the SDK.
- Use RDS Proxy for connection pooling — especially important for Lambda and EKS where connection counts can be high.

```
# Retrieve RDS credentials from Secrets Manager
aws secretsmanager get-secret-value --secret-id prod/db/credentials
```

EC2 to EC2 / Service Communication

- Use private DNS hostnames within a VPC — AWS auto-assigns DNS names like ip-10-0-1-5.ec2.internal.
- For service discovery: Route 53 Private Hosted Zones or AWS Cloud Map.
- Security groups: source security group reference (not IP) for resilient inbound rules.

EKS Pod to AWS Services

- Use IRSA (IAM Roles for Service Accounts) — annotate the K8s ServiceAccount with an IAM Role ARN.
- The pod automatically receives temporary credentials via the projected service account token.
- No access keys in the pod — credentials are transparent to the application.

Connecting to ECS Tasks

ECS Exec allows you to run commands directly in a running ECS container — like kubectl exec for ECS:

```
# Enable ECS Exec on the task definition (enableExecuteCommand: true)
aws ecs update-service \
  --cluster my-cluster \
  --service my-service \
  --enable-execute-command

# Connect to a running ECS container
aws ecs execute-command \
  --cluster my-cluster \
  --task <task-id> \
  --container my-container \
  --interactive \
  --command '/bin/bash'
```

Which Container Registry

ECR (Elastic Container Registry) is the standard for AWS-native deployments. Benefits:

- Integrated with EKS, ECS, Lambda, CodePipeline using IAM authentication — no manual docker login needed.
- Image scanning with Amazon Inspector. Lifecycle policies to auto-delete old images.
- Cross-region and cross-account replication. Private, within the AWS network.

```
# Authenticate Docker to ECR
aws ecr get-login-password | docker login --username AWS \
  --password-stdin 123456789.dkr.ecr.us-east-1.amazonaws.com
```

> *Tip: Use SSM Session Manager or ECS Exec instead of SSH for interactive access — no need to open port 22, all sessions are logged to CloudTrail, and access is controlled by IAM.*

Q21. How do you create AWS Lambda functions and manage the artifacts for deployment? What options do you use to push artifacts to Lambda?

Answer

AWS Lambda functions can be deployed in multiple ways depending on the artifact type (ZIP, container image), size, and deployment pipeline.

Creating a Lambda Function

```
# Create a Lambda function from a ZIP file in S3
aws lambda create-function \
  --function-name my-function \
  --runtime python3.12 \
  --role arn:aws:iam::123456:role/lambda-role \
  --handler app.lambda_handler \
  --code S3Bucket=my-bucket,S3Key=function.zip
```

Artifact Options for Deployment

- Option 1 — ZIP file from S3 (most common for small functions)
- Package code + dependencies into a ZIP. Upload to S3. Deploy directly or via CI/CD.

```
zip -r function.zip . && aws s3 cp function.zip s3://my-bucket/
aws lambda update-function-code \
  --function-name my-function \
  --s3-bucket my-bucket --s3-key function.zip
```

- Option 2 — Container Image from ECR (for large functions or Docker-based workflows)

Lambda supports container images up to 10GB. Push to ECR, then deploy:

```
docker build -t my-lambda .
docker tag my-lambda:latest 123456.dkr.ecr.us-east-1.amazonaws.com/my-lambda:latest
docker push 123456.dkr.ecr.us-east-1.amazonaws.com/my-lambda:latest
aws lambda update-function-code \
  --function-name my-function \
  --image-uri 123456.dkr.ecr.us-east-1.amazonaws.com/my-lambda:latest
```

- Option 3 — Lambda Layers (for shared dependencies)

Extract common libraries into a Layer to keep individual function ZIPs small and avoid duplicating dependencies across functions.

Terraform-Based Deployment

```
resource "aws_lambda_function" "my_function" {
  function_name    = "my-function"
  s3_bucket        = aws_s3_object.function_zip.bucket
  s3_key           = aws_s3_object.function_zip.key
  source_code_hash = filebase64sha256("function.zip")
  runtime          = "python3.12"
  handler          = "app.lambda_handler"
  role             = aws_iam_role.lambda_role.arn
}
```

CI/CD Pipeline for Lambda

- Build: Install dependencies and package into ZIP or Docker image.
- Test: Run unit tests and integration tests.
- Stage: Deploy to a staging Lambda alias. Run smoke tests.
- Promote: Shift traffic via Lambda aliases and weighted routing (canary deployment).

> *Tip: Use Lambda function aliases (dev, staging, prod) and weighted traffic shifting for canary deployments — route 10% of traffic to the new version before promoting to 100%.*

Q22. How do you handle authentication for EKS clusters and store secrets securely in your environment?

Answer

EKS authentication and secret management requires multiple layers: cluster access control, pod-level IAM permissions, and secure secret storage.

EKS Cluster Authentication

EKS uses two authentication mechanisms working together:

- 1. AWS IAM Authentication — Uses the aws-iam-authenticator or aws eks get-token to generate a bearer token from AWS credentials. IAM identity is verified by the Kubernetes API server.
- 2. Kubernetes RBAC — After IAM authentication, RBAC roles determine what the authenticated identity can do within the cluster.

```
# Add cluster access for a new user/role
aws eks update-kubeconfig \
  --name my-cluster --region us-east-1

# Grant cluster access to an IAM role via ConfigMap
kubectl edit configmap aws-auth -n kube-system
```

Add the IAM role mapping:

```
mapRoles:
- rolearn: arn:aws:iam::123456:role/developer-role
  username: developer
  groups: [system:masters]    # or a custom RBAC group
```

EKS Access Entries (Modern Approach — EKS 1.29+)

EKS Access Entries replace the aws-auth ConfigMap. IAM principals are mapped to access policies directly via the EKS API:

```
aws eks create-access-entry \
  --cluster-name my-cluster \
  --principal-arn arn:aws:iam::123456:role/developer-role
```

Pod-Level IAM Authentication — IRSA

IRSA (IAM Roles for Service Accounts) is the secure way for pods to assume IAM roles without access keys:

- Associate an OIDC provider with your EKS cluster.
- Create an IAM Role with a trust policy allowing the cluster's OIDC provider.
- Annotate the Kubernetes ServiceAccount with the Role ARN.
- Pods using that ServiceAccount automatically receive temporary credentials.

```
# Annotate ServiceAccount
kubectl annotate serviceaccount my-sa \
  eks.amazonaws.com/role-arn=arn:aws:iam::123456:role/my-pod-role
```

Secret Storage — Best Practices

- AWS Secrets Manager: Store database passwords, API keys, certificates. Auto-rotate with Lambda rotation functions. Pods retrieve secrets at runtime via the SDK or through the Secrets Store CSI Driver.
- Secrets Store CSI Driver: Mounts Secrets Manager or SSM Parameter Store secrets as files or environment variables into pods — no application code changes needed.
- Sealed Secrets: Encrypt Kubernetes secrets with a cluster-side key. Safe to store encrypted YAML in Git.
- Never use base64 encoded values in plain YAML in Git — Kubernetes Secrets are not encrypted by default.

> *Tip: Enable EKS Envelope Encryption for Kubernetes secrets at rest. EKS uses a KMS Customer Master Key to encrypt the etcd data store — without this, anyone with etcd access can read all secrets in plaintext.*

Q23. Netflix runs multi-cloud. Describe your approach to cross-cloud routing, IAM, and secret syncing.

Answer

Multi-cloud architecture introduces complexity in traffic routing, identity federation, and secret synchronization. The key principle is to keep each cloud's services loosely coupled and independently routable, while maintaining a unified identity and secret layer.

Cross-Cloud Traffic Routing

- Global Load Balancing: Use a cloud-agnostic DNS layer (Route 53 + Azure Traffic Manager + GCP Cloud DNS) to route users to the nearest healthy region regardless of cloud.
- Anycast with Global Accelerator or Cloudflare: Route traffic to the nearest cloud PoP. Cloudflare is fully cloud-agnostic and can proxy to origins in any cloud.
- Service Mesh (Istio / Envoy): Implement a multi-cloud service mesh. Envoy sidecars in each cloud communicate over mTLS. Route traffic across clouds with consistent policies.
- BGP / Direct Connect + ExpressRoute: For private multi-cloud connectivity, use AWS Direct Connect and Azure ExpressRoute connected through a co-location hub (e.g., Equinix).

Cross-Cloud IAM Federation

- OIDC Identity Federation: Use a central Identity Provider (Okta, Google Workspace, Azure AD) as the source of truth. Each cloud trusts this IdP.

- AWS: Configure IAM OIDC provider. Roles can be assumed by federated identities from the IdP.
- Azure: Use Azure AD (Entra ID) as the central IdP. AWS IAM trusts Azure AD via OIDC federation.
- GCP Workload Identity Federation: GCP trusts AWS STS or Azure AD tokens for cross-cloud workload authentication.

```
# AWS IAM trust policy trusting Azure AD
"Principal": {"Federated":
"arn:aws:iam::123:oidc-provider/login.microsoftonline.com/<tenant>"}
"Condition": {"StringEquals": {"oidc:sub": "service-principal-object-id"}}
```

Secret Syncing Across Clouds

- HashiCorp Vault: Cloud-agnostic secret management. A central Vault cluster stores all secrets. Each cloud's workloads authenticate to Vault using their native identity (AWS IAM, Azure Managed Identity, GCP Workload Identity).
- External Secrets Operator: Kubernetes operator that syncs secrets from AWS Secrets Manager, Azure Key Vault, and GCP Secret Manager into Kubernetes Secrets transparently.
- Avoid secret duplication: Store each secret once (in the authoritative cloud or central Vault). Other clouds pull from the source on demand. No copying secrets between clouds.

Key Considerations

- Data residency: Ensure secrets and data don't cross regulatory boundaries.
- Latency: Cross-cloud API calls have higher latency than intra-cloud. Cache credentials and secrets locally with appropriate TTLs.
- Audit trail: Each cloud's CloudTrail / Activity Log / Audit Log must be centralized for unified security monitoring.

> *Tip: For multi-cloud, invest in a platform layer (Kubernetes + service mesh) that abstracts away the underlying cloud. The application should be unaware of which cloud it's running on — only the infra layer knows.*

Q24. Your app teams demand custom AMIs. What's your pre-prod vetting strategy at kernel and runtime?

Answer

Custom AMIs accelerate instance launch time and ensure consistency, but they also introduce risks if not properly vetted. A rigorous AMI pipeline prevents security vulnerabilities from reaching production.

AMI Build Pipeline — Packer

Use HashiCorp Packer to build AMIs as code. The Packer template defines the base OS, packages, agents, and configurations:

```
packer {
  required_plugins { amazon = { source = 'github.com/hashicorp/amazon' } }
}
source "amazon-ebs" "base" {
  ami_name      = "prod-base-{{ timestamp }}"
  instance_type = "m5.large"
  source_ami    = data.aws_ami.amazon_linux_2.id
}
```

Vetting Stage 1 — Security Scanning

- Run Amazon Inspector on the test instance during the Packer build — scans for CVEs in installed packages.
- Use Trivy or Lynis to scan the AMI for OS vulnerabilities, misconfigurations, and CIS benchmark violations.
- Run CIS Benchmark checks (using InSpec or OpenSCAP) to ensure the AMI meets security hardening standards.

```
# Run Trivy against the AMI snapshot
trivy image --exit-code 1 --severity CRITICAL ami://ami-0a1b2c3d4e5f
```

Vetting Stage 2 — Functional Testing

- Launch a test instance from the AMI in an isolated test VPC.
- Run Serverspec or InSpec tests to verify: required agents are installed (CloudWatch Agent, SSM Agent, Fluent Bit), services start correctly, kernel parameters are set as expected.

```
# InSpec test example
describe service('amazon-ssm-agent') do
  it { should be_running }
  it { should be_enabled }
end
```

Vetting Stage 3 — Kernel Validation

- Verify kernel version matches approved version list.
- Check kernel parameters: net.ipv4.tcp_keepalive settings, vm.swappiness, fs.file-max.

- Validate ENA driver (Elastic Network Adapter) is loaded for optimal network performance on newer instance types.

Vetting Stage 4 — Runtime Validation in Staging

- Deploy the new AMI to the staging EKS node group or Auto Scaling Group.
- Run full integration test suite against staging. Monitor for 24 hours.
- Only promote to production AMI after staging validation passes.

AMI Governance

- AMIs older than 90 days are automatically deprecated and removed from the approved list.
- All AMIs tagged with: BuildDate, PipelineRunId, SecurityScanStatus, KernelVersion.
- EC2 Image Builder can automate this entire pipeline as a managed service.

> *Tip: Pin your EKS node group AMI to the validated version. Never use 'latest' AMI IDs in production — an unvetted AMI update can introduce a kernel regression or security vulnerability across your entire fleet.*

Q25. You notice surging NAT costs. No infra changes this week. What could silently trigger that?

Answer

NAT Gateway charges based on the amount of data processed. A sudden surge without infrastructure changes points to unexpected traffic patterns within the VPC. Here are the most common silent culprits:

Most Common Causes

- 1. Application routing S3/ECR traffic through NAT — A new service or updated code is fetching large files from S3 or pulling Docker images from public DockerHub through the NAT Gateway instead of using VPC Endpoints. Solution: Add S3 and ECR VPC Gateway/Interface Endpoints.
- 2. Increased logging volume — Application log level accidentally changed to DEBUG in production. Fluent Bit or CloudWatch Agent shipping 10x more log data than usual through the NAT Gateway to CloudWatch endpoints.
- 3. New batch job or cron — A data pipeline or ETL job started running that fetches large datasets from external sources. Even a single job downloading 100GB daily adds significant NAT cost.
- 4. Increased traffic volume — If your application traffic grew significantly (product launch, viral event), egress through NAT grows proportionally.
- 5. Missing VPC Endpoint — A new AWS service integration (e.g., new use of Secrets Manager, SSM, or CloudWatch) is routing through NAT instead of a VPC endpoint.
- 6. Cross-AZ traffic — Traffic that should stay within an AZ is crossing AZ boundaries, increasing both NAT and data transfer costs.
- 7. VPC Flow Log export to S3 via NAT — VPC Flow Logs being delivered to S3 through NAT rather than through a VPC endpoint.

Investigation Steps

```
# Step 1: Check NAT Gateway metrics in CloudWatch
# BytesOutToDestination — data processed outbound
# PacketsOutToDestination — packet count

# Step 2: Enable VPC Flow Logs and query with Athena
SELECT srcaddr, dstaddr, bytes, protocol
FROM vpc_flow_logs
WHERE action = 'ACCEPT'
ORDER BY bytes DESC LIMIT 20;
```

This query identifies the top talkers — which source IPs are sending the most data through the NAT Gateway and to which destinations.

Fix — Add VPC Endpoints for AWS Services

- S3 Gateway Endpoint: Free. Eliminates all S3 traffic through NAT.
- ECR Interface Endpoints: dkr.ecr, api.ecr — eliminates Docker pull traffic through NAT.
- CloudWatch, SSM, Secrets Manager Interface Endpoints: Keeps AWS API calls within the network.

> *Tip: Run VPC Reachability Analyzer and review VPC Flow Logs regularly. Set a CloudWatch alarm on NAT Gateway BytesOutToDestination — alert if daily bytes exceed a threshold so you catch cost spikes within hours, not at month end.*

Q26. Your EC2 has a public IP and the port is open in the security group, but it's unreachable. Why?

Answer

The existing answer from the source document is correct and complete. Expanding with additional diagnostic context:

Root Cause — Network ACL (NACL)

The most common reason is a Network ACL rule blocking traffic. NACLs are stateless subnet-level firewalls that are evaluated BEFORE security groups. Unlike security groups, NACLs don't automatically allow return traffic — both inbound and outbound rules must explicitly permit the traffic.

```
Security Group: Port 80 ALLOWED from 0.0.0.0/0  ✓
Network ACL:    Inbound Rule 100 - DENY all traffic ✗
Result: Traffic is silently dropped at NACL — unreachable
```

Full Troubleshooting Checklist

- 1. Check NACL rules — Both inbound (for the request) AND outbound (for the response, ephemeral ports 1024-65535) must be allowed.
- 2. Check Route Table — Does the public subnet have a route to the Internet Gateway (0.0.0.0/0 → igw-xxxx)?
- 3. Verify Elastic IP or Public IP — Is the public IP actually assigned to the instance? Check in EC2 console.
- 4. Verify the application is listening — SSH into the instance and run 'ss -tlnp | grep <port>' to confirm the app is bound to 0.0.0.0 (not just 127.0.0.1).
- 5. Check OS firewall — iptables or firewalld on the instance itself may be blocking the port.

```
# Check if app is listening on the correct interface
ss -tlnp | grep ':80'
# Must show 0.0.0.0:80 not 127.0.0.1:80
```

Use VPC Reachability Analyzer

AWS provides a visual tool to trace the exact path and identify where traffic is blocked:

```
aws ec2 start-network-insights-analysis \
  --network-insights-path-id nip-xxxx
```

Tip: NACLs are stateless — this is the #1 gotcha. If you add an inbound ALLOW rule but forget the corresponding outbound rule for response traffic (ephemeral ports 1024-65535), the connection times out silently.

Q27. You shared an AMI with another AWS account, but they still can't launch an instance from it. What's usually missed?

Answer

The existing answer is correct. Expanding with full technical context and steps:

Root Cause — EBS Snapshot Not Shared

An AMI is a metadata object that references one or more EBS snapshots (the actual data). Sharing the AMI gives the target account permission to see and use the AMI definition, but NOT access to the underlying snapshots. When they try to launch, AWS can't copy the snapshot — the launch fails with an error like 'The snapshot does not exist.'

Complete Fix

Share BOTH the AMI and its underlying EBS snapshots with the target account:

```
# Step 1: Find the snapshot IDs associated with the AMI
aws ec2 describe-images --image-ids ami-0a1b2c3d \
  --query 'Images[].BlockDeviceMappings[].Ebs.SnapshotId'

# Step 2: Share the AMI
aws ec2 modify-image-attribute \
  --image-id ami-0a1b2c3d \
  --launch-permission 'Add=[{UserId=TARGET_ACCOUNT_ID}]'

# Step 3: Share each EBS snapshot
aws ec2 modify-snapshot-attribute \
  --snapshot-id snap-0a1b2c3d \
  --attribute createVolumePermission \
  --operation-type add \
  --user-ids TARGET_ACCOUNT_ID
```

If the AMI is Encrypted

If the AMI uses a KMS-encrypted EBS snapshot, you also need to share the KMS key with the target account. The target account must also have permission to use the key to launch the instance:

- Add the target account to the KMS key policy with kms:Decrypt, kms:CreateGrant, kms:DescribeKey.

- Alternatively: re-encrypt the AMI with a new key that is shareable, or use AWS Managed Keys that are auto-shared.

Best Practice — Copy Instead of Share

For production sharing, have the target account COPY the AMI to their own account rather than launching from a shared AMI. This gives them full ownership, avoids key sharing complexity, and allows them to delete/manage the AMI independently:

```
# Run in TARGET account
aws ec2 copy-image \
  --source-region us-east-1 \
  --source-image-id ami-0a1b2c3d \
  --name 'copied-ami-prod'
```

> *Tip: Use EC2 Image Builder with cross-account distribution settings to automatically share both the AMI and its snapshots to target accounts — eliminates the manual sharing mistake entirely.*

Q28. You restored an RDS snapshot for staging, but some queries behave differently than production. Why?

Answer

The existing answer is correct. Expanding with full context and additional causes:

Primary Cause — Default Parameter Group

When you restore an RDS snapshot, AWS assigns the default parameter group for the engine version — not the custom parameter group from the production instance. Parameter groups control hundreds of database settings including query planning, memory, caching, and connection limits.

Common Parameter Differences That Cause Query Behavior Changes

Parameter	Production Setting
work_mem	256MB
shared_buffers	4GB
enable_partitionwise_join	on
max_connections	500
log_min_duration_statement	1000

Full Fix

```
# Step 1: Identify production parameter group
aws rds describe-db-instances \
  --db-instance-identifier prod-db \
  --query 'DBInstances[].DBParameterGroups'

# Step 2: Create matching parameter group for staging
aws rds create-db-parameter-group \
  --db-parameter-group-name staging-pg-match \
  --db-parameter-group-family postgres15

# Step 3: Apply to restored staging instance
aws rds modify-db-instance \
  --db-instance-identifier staging-db \
  --db-parameter-group-name staging-pg-match \
  --apply-immediately
```

Other Differences to Check After Snapshot Restore

- Option Group — RDS Option Groups contain database-level options. These are also not restored from snapshot.
- IAM Authentication — May not be enabled on the restored instance.
- Enhanced Monitoring / Performance Insights — May be disabled by default on restore.
- Security Groups — Restored instance may have a different security group.
- Storage type / IOPS — May revert to gp2 if production uses io1/gp3.

> *Tip: Automate staging provisioning with Terraform — the RDS resource block explicitly sets the parameter group, option group, and security groups. A snapshot restore combined with Terraform apply ensures staging is always configured identically to production.*

Q29. You enabled IAM roles for service accounts in EKS, but your pod can't access S3. The role looks fine. What's the catch?

Answer

The existing answer is correct. Expanding with complete diagnostics and common IRSA failure modes:

Most Common Cause — Incorrect ServiceAccount Annotation

IRSA works by the pod using a ServiceAccount annotated with an IAM Role ARN. If the pod spec doesn't specify serviceAccountName, it defaults to the 'default' ServiceAccount — which has no IAM role annotation.

```
# Check the pod's actual ServiceAccount
kubectl get pod my-pod -o jsonpath='{.spec.serviceAccountName}'

# Check the ServiceAccount's annotation
kubectl get serviceaccount my-sa -n my-namespace -o yaml
```

The annotation must exist:

```
annotations:
  eks.amazonaws.com/role-arn: arn:aws:iam::123456789:role/my-pod-role
```

Complete IRSA Checklist — All Must Be True

- 1. OIDC provider is associated with the EKS cluster:

```
aws eks describe-cluster --name my-cluster \
  --query 'cluster.identity.oidc.issuer'
```

- 2. IAM Role trust policy matches the cluster OIDC provider and the correct ServiceAccount namespace/name:

```
"Condition": {"StringEquals": {
  "oidc.eks.us-east-1.amazonaws.com/id/EXAMPLE:sub":
  "system:serviceaccounts:my-namespace:my-sa"
}}
```

- 3. The ServiceAccount in Kubernetes is annotated with the Role ARN.
- 4. The Pod spec explicitly references the ServiceAccount (serviceAccountName: my-sa).
- 5. The IAM Role has the correct permission policy to access S3.
- 6. The S3 bucket policy (if any) doesn't explicitly deny the role.

Verify Credentials Inside the Pod

```
kubectl exec -it my-pod -- aws sts get-caller-identity
```

If IRSA is working, this will show the IAM Role ARN. If it shows the EC2 node's instance role, IRSA is not configured correctly for this pod.

> *Tip: Use eksctl utils associate-iam-oidc-provider to ensure the OIDC provider is correctly associated. A common mistake is creating the IAM role before the OIDC provider exists — the trust relationship references the wrong issuer URL.*

Q30. ALB is marking your targets as unhealthy, but hitting the app directly works fine. Why?

Answer

The existing answer is correct. Expanding with a complete troubleshooting guide for ALB health check failures:

How ALB Health Checks Work

The ALB sends an HTTP GET request to the health check path on the specified port of each registered target at a configured interval. The target is considered healthy only if it returns an HTTP 200-299 status code within the timeout period.

Common Reasons for Health Check Failures

- 1. Non-200 response — The app returns 301 (redirect), 302, 401, 403, or 404 at the health check path. ALB requires 2xx only. Fix: Change the health check path to a dedicated /health endpoint that returns 200 OK unconditionally.
- 2. Response too slow — Health check timeout is 5 seconds, but the health check endpoint takes 8 seconds to respond (e.g., checks database connectivity). Fix: Increase timeout, or make /health return immediately without external checks.
- 3. Wrong port — Health check is configured for port 80, but the app runs on port 8080. Fix: Update the target group health check port setting.
- 4. Security group blocking ALB — The target's security group doesn't allow inbound traffic from the ALB's security group on the health check port.

- 5. Health check path doesn't exist — The app returns 404 for /health. Fix: Implement the endpoint or change to an existing path.
- 6. VPC routing issue — The ALB is in a different AZ than the targets, and routing is misconfigured.

Check Health Check Configuration

```
aws elbv2 describe-target-health \
  --target-group-arn arn:aws:elasticloadbalancing:...

# Shows reason for unhealthy status:
# Target.ResponseCodeMismatch — wrong HTTP status
# Target.Timeout          — health check timed out
# Target.FailedHealthChecks — multiple consecutive failures
```

Ideal Health Check Endpoint

Implement a dedicated /health endpoint that returns exactly HTTP 200 and a simple JSON body. Do NOT include database or external dependency checks in the health check that ALB uses:

```
app.get("/health", (req, res) => res.status(200).json({ status: "ok" }));
```

Tip: Use separate endpoints: /health for ALB (simple, always fast) and /readiness for Kubernetes probes (can check DB connectivity). ALBs don't support Kubernetes readiness concepts — keep ALB health checks as lightweight as possible.

Q31. You pushed a new image to ECR and updated your ECS task definition, but it still runs the old version. Why?

Answer

The existing answer is correct. Expanding with all possible causes and fixes:

Primary Cause — Image Tag Immutability and Caching

If using the 'latest' tag (or any mutable tag), ECS checks if the tag is already pulled on the container instance. If it is, it doesn't re-pull — it uses the cached layer. The container starts with the old image even though the registry has a newer one.

Complete List of Causes

- 1. Mutable image tags (latest) — ECS caches 'latest' at pull time. Subsequent deployments with the same tag don't trigger a new pull. Fix: Use immutable image tags (git commit SHA or build number) — a new tag always forces a pull.

```
# Bad:  image: 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
# Good: image: 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:abc1234
```

- 2. ECS Service not updated — The task definition was updated but the ECS Service was not forced to redeploy. Task definitions in ECS are immutable — creating a new revision doesn't automatically restart running tasks.

```
# Force new deployment after updating task definition
aws ecs update-service \
  --cluster my-cluster \
  --service my-service \
  --task-definition my-task:NEW_REVISION \
  --force-new-deployment
```

- 3. Minimum healthy percent — ECS rolling update keeps old tasks running if new tasks don't become healthy. If the health check fails on the new task, ECS rolls back to the old version.
- 4. Service quota / insufficient capacity — New tasks can't start due to insufficient EC2 capacity or Fargate quota limits. Old tasks continue running.
- 5. Incorrect task definition ARN — CI/CD pipeline updated the wrong task definition revision or the wrong service.

Verify Which Task Definition is Running

```
aws ecs describe-services \
  --cluster my-cluster --services my-service \
  --query 'services[].taskDefinition'

aws ecs describe-tasks \
  --cluster my-cluster --tasks $(aws ecs list-tasks \
  --cluster my-cluster --service-name my-service \
  --query 'taskArns[0]' --output text) \
  --query 'tasks[].containers[].image'
```

Tip: Always use immutable image tags tied to the Git commit SHA in CI/CD pipelines. Tools like docker build --build-arg GIT_SHA=$(git rev-parse --short HEAD) make this straightforward and give you full traceability.

Q32. In EKS, your stateful pod using an EBS volume is stuck in Pending. Why doesn't it reschedule?

Answer

The existing answer is correct. Expanding with full diagnostic steps and prevention strategies:

Root Cause — EBS AZ Affinity

EBS volumes are Availability Zone-specific — a volume created in us-east-1a cannot be attached to a node in us-east-1b. When EKS tries to reschedule a pod on a node in a different AZ, the EBS CSI driver cannot attach the volume and the pod remains Pending.

```
kubectl describe pod my-stateful-pod
# Events section will show:
# Warning: FailedScheduling  pod has unbound immediate PersistentVolumeClaims
# Warning: FailedAttachVolume  multi-attach error for volume: ...
#          EBS volumes can only be attached to one node at a time
```

Full Diagnostic Steps

- Step 1: Identify the AZ of the EBS volume:

```
kubectl get pvc my-pvc -o jsonpath='{.spec.volumeName}'
aws ec2 describe-volumes --volume-ids vol-xxxx \
  --query 'Volumes[].AvailabilityZone'
```

- Step 2: Check if any nodes exist in that AZ:

```
kubectl get nodes -L topology.kubernetes.io/zone
```

- Step 3: Check if the pod has node affinity that conflicts with volume AZ:

```
kubectl describe pod my-pod | grep -A5 'Node-Selectors\|Tolerations\|Affinity'
```

Fixes

- Fix 1 — Ensure node group covers the same AZ as the volume. If using Karpenter, add AZ constraints to the NodePool.
- Fix 2 — Use WaitForFirstConsumer binding mode in StorageClass. This delays PV creation until the pod is scheduled, ensuring the volume is created in the same AZ as the node:

```
kind: StorageClass
metadata:
  name: gp3-wait
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
```

- Fix 3 — Use EFS instead of EBS for workloads that need multi-AZ access. EFS is a managed NFS that spans all AZs and can be mounted from any node simultaneously.
- Fix 4 — If pod must use EBS, pin it to the correct AZ using nodeAffinity:

```
affinity:
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - key: topology.kubernetes.io/zone
          operator: In
          values: [us-east-1a]
```

Tip: For StatefulSets with EBS, set volumeBindingMode: WaitForFirstConsumer in the StorageClass — this is the single most important setting to prevent AZ-binding issues. Make it the default StorageClass in your cluster.

Q33. Create a Python script for parsing and analyzing AWS CloudWatch metrics with data visualization.

Answer

This coding question demonstrates your ability to programmatically interact with AWS services and present data meaningfully. Below is a complete script that fetches CloudWatch metrics and generates a visualization.

Script — CloudWatch Metrics Analyzer

```
import boto3
import json
from datetime import datetime, timedelta
from collections import defaultdict

def get_cloudwatch_metrics(namespace, metric_name, dimensions, hours=24):
    """Fetch CloudWatch metric data for the specified period."""
    client = boto3.client('cloudwatch', region_name='us-east-1')
```

```
    end_time = datetime.utcnow()
    start_time = end_time - timedelta(hours=hours)

    response = client.get_metric_statistics(
        Namespace=namespace,
        MetricName=metric_name,
        Dimensions=dimensions,
        StartTime=start_time,
        EndTime=end_time,
        Period=3600,      # 1-hour granularity
        Statistics=['Average', 'Maximum', 'Minimum']
    )
    return sorted(response['Datapoints'], key=lambda x: x['Timestamp'])

def analyze_metrics(datapoints):
    """Calculate summary statistics from metric data points."""
    if not datapoints: return {}
    averages = [dp['Average'] for dp in datapoints]
    return {
        'count':    len(averages),
        'mean':     round(sum(averages) / len(averages), 2),
        'maximum':  round(max(dp['Maximum'] for dp in datapoints), 2),
        'minimum':  round(min(dp['Minimum'] for dp in datapoints), 2),
        'latest':   round(datapoints[-1]['Average'], 2)
    }

def print_chart(datapoints, metric_name, unit='%', width=50):
    """Print an ASCII bar chart of metric values over time."""
    if not datapoints: return
    max_val = max(dp['Average'] for dp in datapoints) or 1
    print(f'\n  {metric_name} over time (ASCII chart)')
    print('  ' + '—' * (width + 20))
    for dp in datapoints:
        val = dp['Average']
        bar_len = int((val / max_val) * width)
        bar = '' * bar_len
        ts = dp['Timestamp'].strftime('%H:%M')
        print(f'  {ts} | {bar:<{width}} {val:.1f}{unit}')
    print('  ' + '—' * (width + 20))

def main():
    instance_id = 'i-0a1b2c3d4e5f67890'  # Replace with your instance ID
    metrics = [
        ('AWS/EC2', 'CPUUtilization',     '%'),
        ('AWS/EC2', 'NetworkIn',           'Bytes'),
        ('AWS/EC2', 'StatusCheckFailed',   'Count'),
    ]
    dimensions = [{'Name': 'InstanceId', 'Value': instance_id}]

    print(f'CloudWatch Metrics Report')
    print(f'Instance: {instance_id}')
    print(f'Period:   Last 24 hours')
    print('=' * 60)

    for namespace, metric_name, unit in metrics:
        data = get_cloudwatch_metrics(namespace, metric_name, dimensions)
        stats = analyze_metrics(data)
        print(f'\n  {metric_name}')
        for k, v in stats.items():
            print(f'    {k:10}: {v} {unit}')
        print_chart(data, metric_name, unit)

if __name__ == '__main__':
    main()
```

Running the Script

```
# Install dependencies
pip install boto3

# Ensure AWS credentials are configured
```

```
export AWS_PROFILE=my-profile
python cloudwatch_analyzer.py
```

Extending for Real Visualization

For production-grade charts, replace print_chart with matplotlib or output JSON to a dashboard:

```
import matplotlib.pyplot as plt
timestamps = [dp['Timestamp'] for dp in data]
values     = [dp['Average']   for dp in data]
plt.plot(timestamps, values, marker='o')
plt.title(f'{metric_name} - Last 24 Hours')
plt.savefig(f'{metric_name}.png')
```

> *Tip: In production, use CloudWatch dashboards or Grafana with the CloudWatch data source instead of custom scripts for real-time visualization. Scripts like this are ideal for ad-hoc analysis, reporting, or CI/CD integration testing.*
